

# Stacks and Queues

From C++ adapters, algorithms to runtime memory, stack unwinding

---

## Core abstractions

LIFO vs FIFO, invariants, and operation semantics

## C++ standard library

container vs. adapters, `std::stack`, `std::queue`, `std::deque`

## Application

DFS/BFS, parsers, schedulers, queues in low-latency systems, exception unwinding

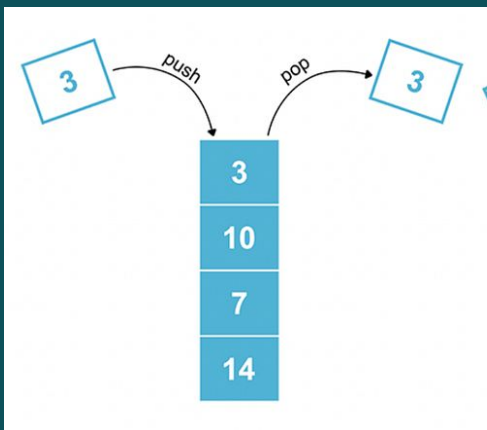
# Concept overview

## Stack = LIFO

The most recently inserted element is the first one removed.

Think: plates on a spring-loaded tray, function calls, undo history, expression evaluation.

Supported functions: push(), pop(), top(), size()

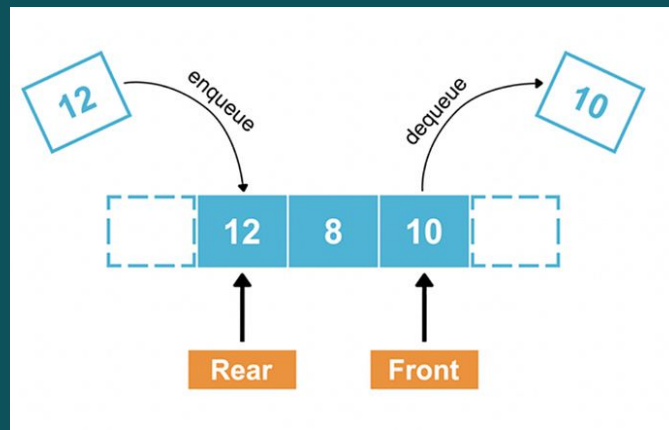


## Queue = FIFO

The earliest inserted element is the first one removed.

Think: ticket lines, task scheduling, network buffers, breadth-first search.

Supported functions: push(), pop(), front(), back(), size()



# Container vs. adapter in C++

Why `std::stack` and `std::queue` feel “limited”

## Container

Owns and stores elements directly.

Examples: `std::vector`, `std::deque`, `std::list`.

Containers expose rich operations: iteration, insertion, erasure, indexing (when supported), allocator behavior, and invalidation rules.



## Adapter

Wraps another container and intentionally exposes only a restricted interface.

`std::stack` and `std::queue` are container adapters. They trade flexibility for safety + semantic clarity.

You cannot iterate a `std::stack` because the abstraction wants you to think in LIFO, not “random traversal.”

**Adapters are not new storage structures; they are controlled views over an underlying container.**

# Source Code

Defined in header `<stack>`

```
template<
    class T,
    class Container = std::deque<T>
> class stack;
```

Defined in header `<queue>`

```
template<
    class T,
    class Container = std::deque<T>
> class queue;
```

## Things to Note

The default container for both stack and queue are deques, but can be parameterized to vector (for stack), list, etc

The container must support specific operations, depending on the type of adapter.

# Why `std::deque` is the default

## What deque gives you

- Fast insertion/removal at both ends
- Random access by index
- No need to shift elements like a vector-based front-pop would
- Better fit than vector for queues, and a fine default for stacks

Even though insertion on both sides is not needed for `std::stack`, the fact that it avoids re-allocation copying compared to `std::vector` and has a better cache-locality compared to `std::list` makes it a better choice overall

## For `std::stack`

Needs `back()`, `push_back()`, and `pop_back()`.  
`std::deque`, `std::vector`, and `std::list` can work.

## For `std::queue`

Needs `front()`, `back()`, `push_back()`, and `pop_front()`.  
`std::deque` and `std::list` work;  
`std::vector` does not.



## **deque = double-ended queue**

Efficient growth/shrink at both ends is exactly what a queue needs.

# Memory stack vs. DSA stack

## Memory stack / runtime call stack

A language/runtime mechanism. Each call pushes a frame containing a return address, parameters, saved registers, and local state. Returning from the function pops that frame.

This is about program execution and storage lifetime, happens at **runtime**

## DSA stack

An abstract data type with push / pop / top. You can implement it with an array, `std::vector`, `std::deque`, or a linked list.

```
Breakpoint 1, processBuy (incoming=...) at matchengine.cpp:155
155     void processBuy(Order incoming) {
(gdb) bt
#0  processBuy (incoming=...) at matchengine.cpp:155
#1  0x0000555555558b25 in processOrder (o=...) at matchengine.cpp:222
#2  0x0000555555558cd0 in main () at matchengine.cpp:245
(gdb) █
```

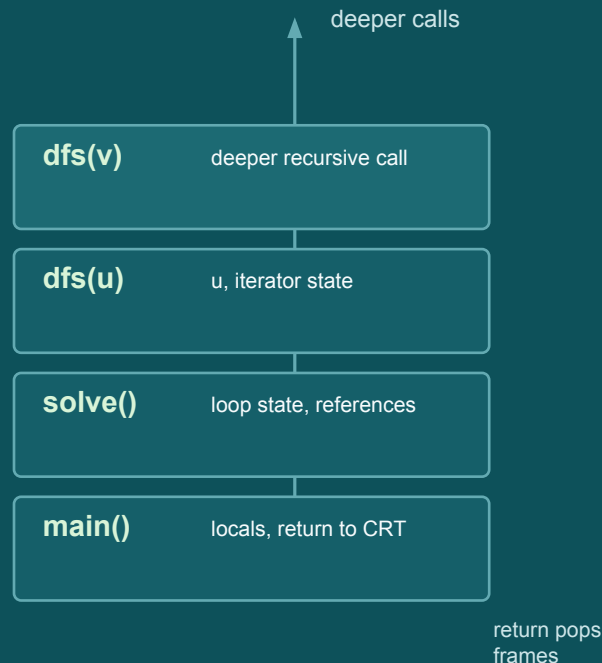
# What the runtime stack is really doing

Call frames, recursion depth, and unwinding

- A recursive algorithm often uses the runtime call stack implicitly.
- Deep recursion can overflow the call stack even if the algorithm is logically correct.
- Iterative versions with an explicit stack move control from “runtime-managed” to “programmer-managed.”

recursive DFS uses the memory stack

```
1 void dfs(int u) {  
2     visited[u] = true;  
3     for (int v : adj[u]) {  
4         if (!visited[v]) dfs(v); // pushes a  
new call frame  
5     }  
6 }
```



# Stack unwinding

What happens when control leaves nested calls because of an exception

RAII during exception propagation

```
1 struct Guard {
2     string name;
3     ~Guard() { cerr << "destroy " << name << "\n"; }
4 };
5
6 void g() {
7     Guard c{"g"};
8     throw runtime_error("boom");
9 }
10
11 void f() {
12     Guard b{"f"};
13     g();
14 }
15
16 int main() {
17     try {
18         Guard a{"main"};
19         f();
20     } catch (const exception& e) {}
21 }
```

- When g() throws, control moves upward until a matching catch is found.
- As frames exit, local objects are destroyed in reverse order.
- That cleanup process is stack unwinding — one reason RAII is so powerful in C++.

**Observed destruction order:**  
**g -> f -> main**

## Why this matters

Unwinding connects the runtime call stack to language semantics: exceptions, destructors, noexcept behavior, and resource safety.



# std::stack deep dive

## basic usage

```
1 #include <stack>
2 #include <vector>
3
4 std::stack<int> a;    // default: deque<int>
5 std::stack<int, std::vector<int>> b;
6 a.push(10);
7 a.push(20);
8 int x = a.top();    // 20
9 a.pop();            // removes 20
```

- `top()` reads the last inserted element without removing it.
- `push()` / `emplace()` append to the underlying container's back.
- `pop()` removes from the back and does not return the removed value.
- You must call `top()` before `pop()` if you want the element.

## Common gotcha

This is invalid thinking: `int x = s.pop();`  
C++ `pop()` returns void. The correct pattern is: `x = s.top(); s.pop();`

**A stack is ideal when the newest work item must be handled first.**

# std::queue deep dive

## basic usage

```
1 #include <queue>
2
3 std::queue<int> q;
4 q.push(10);
5 q.push(20);
6 q.push(30);
7
8 int first = q.front(); // 10
9 int last  = q.back();  // 30
10 q.pop();              // removes 10
```

- front() observes the next element to leave the queue.
- back() observes the most recently inserted element.
- push() adds at the back; pop() removes from the front.
- A queue preserves arrival order and is natural for scheduling and BFS.

## Why vector is not a queue backend

pop\_front() is not supported - If you pop from the front of a vector, you would need to shift later elements or simulate a circular buffer yourself.

**Queue discipline models “fairness by arrival order.”**

# Method complexities and backend tradeoffs

Complexity belongs to the adapter + its underlying container

| Structure               | Operation                                                     | Typical cost               | Notes                                         |
|-------------------------|---------------------------------------------------------------|----------------------------|-----------------------------------------------|
| <code>std::stack</code> | <code>top()</code>                                            | $O(1)$                     | Accesses the back of the underlying container |
| <code>std::stack</code> | <code>push()/emplace()</code>                                 | $O(1)$ amortized or $O(1)$ | Depends on backend; vector may reallocate     |
| <code>std::stack</code> | <code>pop()</code>                                            | $O(1)$                     | Removes from the back                         |
| <code>std::queue</code> | <code>front()/back()</code>                                   | $O(1)$                     | Observe both ends                             |
| <code>std::queue</code> | <code>push()/emplace()</code>                                 | $O(1)$                     | Adds at the back                              |
| <code>std::queue</code> | <code>pop()</code>                                            | $O(1)$                     | Removes from the front                        |
| <code>std::deque</code> | <code>push_front/back</code> ,<br><code>pop_front/back</code> | $O(1)$                     | Designed for efficient end operations         |
| <code>std::deque</code> | <code>operator[]</code>                                       | $O(1)$                     | Random access supported                       |

# Algorithm applications

## Stack patterns

- Depth-first search
- Backtracking
- Expression parsing and evaluation
- Undo / redo
- Balanced parentheses
- Monotonic stack for next greater / smaller element
- Manual simulation of recursion

## Queue patterns

- Breadth-first search
- Level-order tree traversal
- Task scheduling
- Producer-consumer pipelines
- Rate limiting / buffering
- Shortest path in unweighted graphs
- Topological processing (Kahn-style indegree queue)

## Ask yourself

Do I want the newest unfinished work first? -> stack

Do I want the earliest arrival first?  
-> queue

Do I need priority by score or time?  
-> maybe priority\_queue instead

# Code walkthrough: Matching Parentheses with stack

Single traversal storing unmatched brackets with a stack

## Valid Parentheses

```
1 bool balanced(const string& s) {
2     stack<char> st;
3
4     for (char c : s) {
5         if (c == '(') {
6             st.push(c);
7         } else if (c == ')') {
8             if (st.empty()) return false;
9             st.pop();
10        }
11    }
12
13    return st.empty();
14 }
```

- The stack stores unmatched opening parentheses.
- If the stack is empty when a closer arrives, the string is invalid immediately.
- After the scan, an empty stack means every opener found a match.

## Big conceptual win

Nested structure is naturally LIFO: the last unmatched opener must be the first one closed.

# Code walkthrough: BFS with a queue

FIFO preserves distance layers

## breadth-first search

```
1 vector<int> bfs_dist(const vector<vector<int>>& adj, int
src) {
2     vector<int> dist(adj.size(), -1);
3     queue<int> q;
4     dist[src] = 0;
5     q.push(src);
6
7     while (!q.empty()) {
8         int u = q.front();
9         q.pop();
10        for (int v : adj[u]) {
11            if (dist[v] != -1) continue;
12            dist[v] = dist[u] + 1;
13            q.push(v);
14        }
15    }
16    return dist;
17 }
```

- Nodes are processed in order of discovery.
- Because every edge has equal weight here, the first time you reach a node is along a shortest path.
- That distance-layer property comes directly from FIFO order.

### Mental model

A queue expands the search frontier one wave at a time. That is why BFS feels “layered” while DFS feels “deep.”

# Beyond basic DFS/BFS: interview-grade patterns

Where stacks and queues become technique, not just containers

## monotonic stack

```
1 // Monotonic increasing stack for next smaller
  element
2 vector<int> next_smaller(vector<int> a) {
3     vector<int> ans(a.size(), -1);
4     stack<int> st; // stores indices
5     for (int i = 0; i < (int)a.size(); ++i) {
6         while (!st.empty() && a[i] <
a[st.top()]) {
7             ans[st.top()] = a[i];
8             st.pop();
9         }
10        st.push(i);
11    }
12    return ans;
13 }
```

## Monotonic stack insight

The stack is not storing “all history”; it stores only candidates that still matter. This turns many nested-loop problems into linear-time scans.

## Deque insight

Sliding-window maximum/minimum usually uses a deque, because you need both: push/pop at the back and eviction from the front.

## Queue pattern

Multi-source BFS starts by pushing many sources at distance 0 before the wavefront begins.

# Application in HFT / low-latency systems

throughput, fairness, and cache behavior

## Where queues appear

- NIC receive/transmit rings
- Message passing between stages
- Order/event buffering
- Work queues in thread pools
- Backpressure boundaries between producers and consumers

Lock-free queues: atomic operations replace mutexes;  
Compare-And-Swap (CAS): update a value if and only if it had not been changed by another thread.

## Design pressure

In a hot path, generic containers may lose to a fixed-size circular buffer because predictable memory layout, bounded capacity, and fewer allocations matter.

But the abstract behavior is still a queue.

## Where stacks appear

Function-call stack is unavoidable for execution.  
Explicit stacks also show up in parsers, state machines, and backtracking logic.

In low-latency code, deep recursion is often avoided to keep control predictable.



# What to remember

Choosing well is mostly about modeling the order of work

## Use a stack when...

- Newest item should be handled first
- You are modeling nested structure
- You need backtracking or undo
- You want an explicit replacement for recursion

## Use a queue when...

- Oldest item should be handled first
- You are modeling arrivals or fairness
- You want layered exploration (BFS)
- You have producer-consumer flow

## Use `std::deque` when...

- You need operations at both ends
- You want a backend for stack/queue adapters
- You need random access without committing to vector semantics